

CHAPTER 4

Software Testing Fundamentals

Coverage

12 Sections

Course

Software Engineering

Deck

56 Slides

Topics

Levels · Types · Docs ·
Environments

Chapter 4 — Table of Contents

4.1 Introduction to Software Testing

4.2 Verification and Validation

4.3 Principles of Software Testing

4.4 Levels of Testing

4.5 Types of Testing

4.6 Functional Testing

4.7 Non-Functional Testing

4.8 Regression Testing

4.9 Smoke and Sanity Testing

4.10 Test Environment Setup

4.11 Test Documentation

4.12 Chapter Summary

4.1

Introduction to Software Testing

Why testing is the cornerstone of software quality.

Chapter 4 — Software Testing Fundamentals

Section 4.1

What is Software Testing?

Software Testing (IEEE Std 829):

The process of executing a program or system with the intent of finding errors. It is a planned, systematic activity to evaluate software against specified requirements and to detect defects before delivery to users.

**Find Defects**

Discover faults in the software before users encounter them in production.

**Verify Requirements**

Confirm the software implements all specified requirements correctly.

**Validate User Needs**

Ensure the software satisfies real user expectations and business needs.

**Build Confidence**

Provide stakeholders with evidence that the software is fit for release.

**Prevent Defects**

Identify root causes of defects to prevent similar issues in future.

**Measure Quality**

Provide objective metrics about software quality for management decisions.

✂ Testing cannot PROVE software is correct — It can only find defects. Dijkstra (1972): 'Testing can show the presence of bugs, not their absence.'

Section 4.1

Why Testing is Critical — Real Failures

Famous software failures that could have been prevented by thorough testing:

Mars Climate Orbiter (1999)

Unit mismatch: one team used imperial, another metric. \$327M mission lost. Root cause: no integration testing of unit conversions.

Therac-25 (1985–87)

Race condition in software caused massive radiation overdoses killing at least 3 patients. Root cause: inadequate safety testing.

Ariane 5 Rocket (1996)

64-bit float to 16-bit int overflow caused explosion. \$500M lost in 40 seconds. Root cause: no testing of reused component in new context.

Knight Capital (2012)

Faulty deployment activated old trading algorithm. \$440M lost in 45 minutes. Root cause: no regression testing of deployment.

Healthcare.gov (2013)

US healthcare website crashed on launch day under real load. Root cause: no load testing or performance testing conducted.

Facebook Global Outage (2021)

BGP routing configuration error took Facebook offline for 6 hours (~\$60M loss). Root cause: insufficient configuration change testing.



A thorough testing strategy with unit, integration, regression, and performance testing would have prevented all these failures. Testing = risk management.

Section 4.1

Testing vs Debugging — Key Distinction

Testing

- Systematic process to FIND defects
- Executed by testers (or automated)
- Planned, documented activity
- Follows test plan and test cases
- Shows PRESENCE of defects
- Can be done without source code
- Output: defect reports, test results

Debugging

- Process to LOCATE and FIX defects
- Executed by developers
- Usually unplanned, reactive activity
- Follows error symptoms and logs
- RESOLVES defects identified in testing
- Requires access to source code
- Output: fixed code, updated build

✂ Testing finds WHERE something is wrong. Debugging finds WHY it is wrong and fixes it. Both are essential but are distinct activities with

4.2

Verification and Validation

V&V — two fundamental quality assurance concepts.

Section 4.2

Verification vs Validation

“ *Verification: 'Are we building the product RIGHT?' Validation: 'Are we building the RIGHT product?'*

— Barry Boehm

Verification

Question:	Are we building the product RIGHT?
What:	Process-oriented evaluation — checks that deliverables conform to specifications, standards, and plans.
When:	During development — at end of each phase
How:	Reviews · Inspections · Walkthroughs · Audits · Static analysis
Example:	Checking SRS against IEEE standard format · Design review against requirements

Validation

Question:	Are we building the RIGHT product?
What:	Product-oriented evaluation — confirms the final product meets user needs and intended use.
When:	At end of development — against user needs
How:	Dynamic testing · User acceptance testing · System testing
Example:	End-users test if the system actually solves their business problem

4.3

Principles of Software Testing

Seven universal principles from ISTQB.

Chapter 4 — Software Testing Fundamentals

Section 4.3

7 Principles of Software Testing (ISTQB)

These seven principles (ISTQB Foundation, Glenford Myers) guide all professional software testing:

1

Testing Shows Presence of Defects

Testing can show defects ARE present, but cannot prove defects are ABSENT. Even 100% coverage cannot guarantee defect-free software.

2

Exhaustive Testing is Impossible

It is impossible to test all input combinations, paths, and scenarios. Risk assessment and prioritisation guide where to focus testing effort.

3

Early Testing Saves Money

Testing activities should begin as early as possible. The earlier a defect is found, the cheaper it is to fix (1x at requirements → 100x in production).

4

Defect Clustering

A small number of modules (typically 20%) contain the majority of defects (80%). Testing effort should focus on historically defect-prone areas.

5

Pesticide Paradox

Repeating the same test cases will not find new bugs — tests become immune like pesticides. Regularly review and update test cases to find new defects.

6

Testing is Context Dependent

Testing is done differently in different contexts. Safety-critical testing (medical, aviation) requires a very different approach than testing a website.

7

Absence of Errors Fallacy

Finding and fixing defects is pointless if the system built is unusable or does not meet user needs. A bug-free system that does the wrong thing is still a failure.

4.4

Levels of Testing

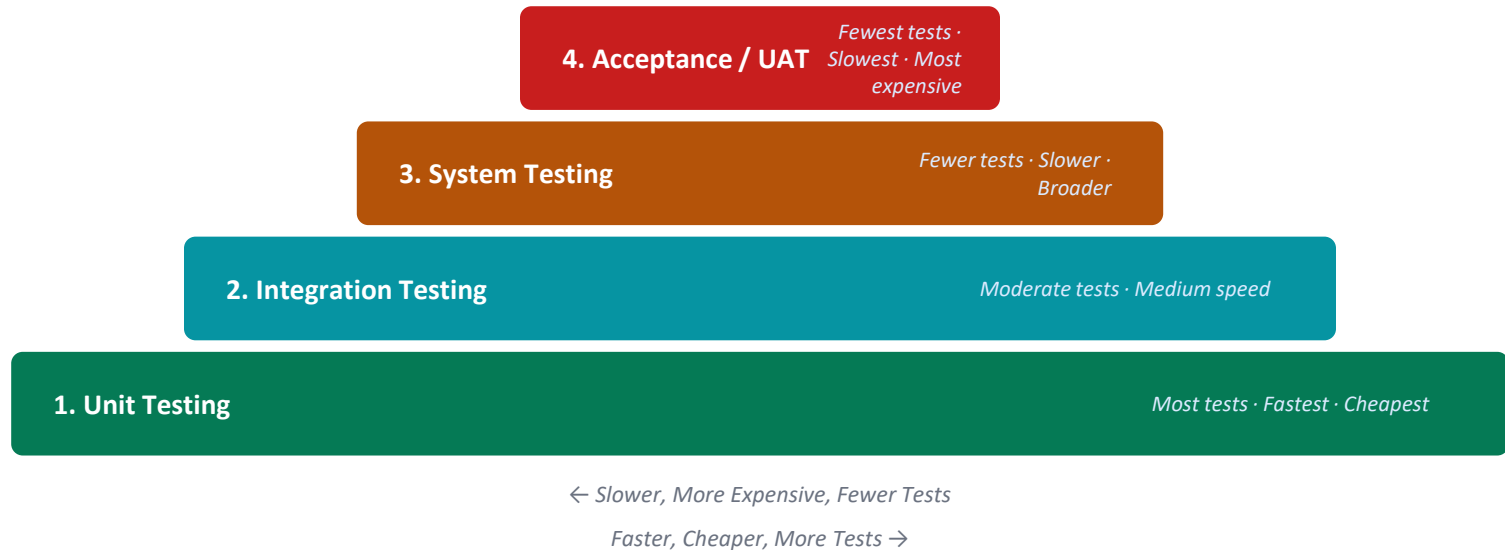
Unit · Integration · System · Acceptance — the test pyramid.

Chapter 4 — Software Testing Fundamentals

Section 4.4

Test Pyramid — Overview

The Test Pyramid (Mike Cohn) shows the optimal distribution of tests by level:



Ideal distribution: ~70% unit tests · ~20% integration tests · ~10% E2E/acceptance. High confidence at lowest cost.

Section 4.4

Level 1 — Unit Testing

Unit testing tests the smallest testable parts (units) of software in isolation from the rest of the system:

What is tested

Individual functions, methods, procedures, or classes in isolation

Who tests

Developers (during or after coding) — TDD: before coding

When tested

During implementation phase — immediately after writing the unit

Approach

White-box testing — requires access to source code and internal logic

Test basis

Design specifications, code structure, module design documents

Typical tools

JUnit (Java), pytest (Python), NUnit (.NET), Jest (JavaScript), xUnit

Coverage goals

Statement coverage: 80%+ · Branch coverage: 70%+ · Path coverage: key paths

✓ **Benefits:** Fast feedback · Catches defects early · Enables safe refactoring · Documents behaviour

✗ **Limitations:** Cannot test interactions · Does not find integration bugs · Requires stubs/mocks

Section 4.4

Levels 2–4: Integration, System & Acceptance Testing

2. Integration Testing

Who:	Dev / Test team
Tests:	Tests interactions between integrated units/modules. Verifies data flows, API contracts, and component interfaces.
When:	After unit testing — as modules are combined
Method:	Black-box or grey-box · Top-down · Bottom-up · Big-bang · Sandwich
Tools:	REST Assured, Postman, SoapUI, Mockito, WireMock

3. System Testing

Who:	Independent QA team
Tests:	Tests the complete integrated system against functional and non-functional requirements. Treats system as a black box.
When:	After all modules integrated and unit/integration tested
Method:	Black-box testing · Equivalence partitioning · Boundary value analysis
Tools:	Selenium, Cypress, TestNG, JMeter, OWASP ZAP

4. Acceptance Testing

Who:	Customer / End users
Tests:	Validates the system meets business needs and user expectations. Final gate before production deployment.
When:	After system testing — pre-production / go-live decision
Method:	UAT (User Acceptance Test) · Alpha/Beta testing · Contract testing
Tools:	Manual scripts · Cucumber/SpecFlow (BDD) · TestRail

✦ Each testing level has a corresponding V-Model development phase. Always define exit criteria for each level before proceeding to the

4.5

Types of Testing

Black-box · White-box · Grey-box testing approaches.

Section 4.5

Black-Box, White-Box & Grey-Box Testing

Black-Box Testing

Tests software without knowledge of internal code structure. Only inputs and outputs are considered. Tester sees only the 'black box' — no internal visibility.

Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing

Pros:

- Unbiased — independent of implementation
- Tests from user perspective

Cons:

- Cannot test all code paths
- May miss error conditions in code

White-Box Testing

Tests software with full knowledge of internal code. Tests internal logic, paths, branches, and conditions. Also called glass-box, structural, or clear-box testing.

Techniques:

- Statement Coverage
- Branch Coverage
- Path Coverage

Pros:

- Can achieve 100% code coverage
- Finds unused code and dead code

Cons:

- Requires programming expertise
- Very expensive at system level

Grey-Box Testing

Combines black-box and white-box techniques. Tester has partial knowledge of internals — enough to design better tests without full code access.

Techniques:

- Integration testing
- Regression testing
- Penetration testing

Pros:

- Balance of both approaches
- Non-intrusive but more targeted

Cons:

- Not as thorough as white-box
- Requires some technical knowledge

4.6

Functional Testing

Testing WHAT the software does against requirements.

Section 4.6

Functional Testing — Overview & Techniques

Functional testing verifies that each function of the software operates in conformance with the requirement specification (black-box approach):

1 Equivalence Partitioning

Divide input data into valid and invalid partitions. Test one value from each partition. Reduces number of test cases while maintaining coverage.

Example: Age field accepts 18–65. Partitions: <18 (invalid), 18–65 (valid), >65 (invalid). Test: 15, 30, 70.

2 Boundary Value Analysis

Focus test cases on boundary values where errors most commonly occur. Test the minimum, maximum, and just above/below boundaries.

Example: Age 18–65. Test: 17, 18, 19, 64, 65, 66 — boundary values on both edges.

3 Decision Table Testing

Create a table of conditions and actions. Systematically cover all combinations of input conditions. Good for business logic with multiple conditions.

Example: Loan approval depends on: income>50K AND credit>700 AND employed.

4 State Transition Testing

Test transitions between system states. Identify all valid/invalid transitions, events, and actions.

Example: Online order: New→Confirmed→Shipped→Delivered. Test all transitions and invalid ones (e.g., Delivered→New).

4.7

Non-Functional Testing

Testing HOW WELL the software performs.

Section 4.7

Non-Functional Testing — Overview

Non-functional testing evaluates HOW WELL the software performs its functions — not WHAT it does:

⚡ Performance Testing

Tests how the system performs under expected and peak load. Measures response time, throughput, and resource utilisation.

Sub-types: Load Testing — normal expected load · Stress Testing — beyond normal capacity

Tools: JMeter+

🔒 Security Testing

Identifies vulnerabilities, threats, and risks in the software. Ensures confidentiality, integrity, and availability of data.

Sub-types: Penetration Testing (Pen Test) · Vulnerability Scanning — SAST/DAST

Tools: OWASP ZAP+

👤 Usability Testing

Evaluates how easy and intuitive the software is to use. Tests user experience, learnability, and satisfaction.

Sub-types: User interface consistency · Navigation and workflow testing

Tools: UserTesting.com+

🔒 Reliability Testing

Tests the software's ability to perform consistently and correctly over time without failure.

Sub-types: MTBF (Mean Time Between Failures) · MTTR (Mean Time To Repair)

Tools: Chaos Engineering: Chaos Monkey+

📱 Compatibility Testing

Tests whether software works correctly across different browsers, devices, OS, and network conditions.

Sub-types: Cross-browser testing (Chrome/Firefox/Safari) · Mobile device compatibility testing

Tools: BrowserStack+

🔧 Maintainability Testing

Tests how easy the software is to maintain, modify, and understand. Measures code quality and technical debt.

Sub-types: Code complexity (cyclomatic complexity) · Code coverage measurement

Tools: SonarQube+

👉 Non-functional testing is often neglected until after launch, causing performance crashes, security breaches, and accessibility lawsuits.

Section 4.7

Performance Testing — Deep Dive

Performance testing is the most commonly required non-functional test. Understanding the four main types is essential:

Load Testing

Tests system behaviour under EXPECTED normal load. Verifies response times and throughput meet SLA requirements.

Example: Simulate 1,000 concurrent users. Verify response time < 2 seconds. Measure throughput (transactions per second).

Use when: Before release and after major changes. Part of performance baseline.

Stress Testing

Tests system behaviour BEYOND normal peak capacity until it breaks or degrades. Identifies the breaking point.

Example: Start at 1,000 users, increase by 500 until system degrades. Identify maximum capacity and failure mode.

Use when: To understand system limits and failure behaviour. Disaster planning.

Endurance (Soak) Testing

Tests system under expected load for an EXTENDED PERIOD to detect memory leaks, database connection issues.

Example: 1,000 concurrent users for 72 hours. Monitor memory usage, response time, database connections over time.

Use when: Before major releases and after performance improvements.

Spike Testing

Tests system response to SUDDEN dramatic increases in load — simulates flash crowds or viral events.

Example: Instantly go from 100 to 5,000 users. Verify system handles the spike and recovers gracefully afterwards.

Use when: E-commerce (Black Friday), event ticketing systems, news websites.

4.8

Regression Testing

Ensuring changes do not break existing functionality.

Section 4.8

Regression Testing — Definition & Purpose

Regression testing re-executes tests on modified software to confirm that previously working functionality has not been broken by new changes:

When to Run Regression Tests

- After any code change or bug fix
- After adding new features
- After configuration changes
- After database schema changes
- After third-party library updates
- After performance optimisations
- After every sprint (Agile) or release

✓ Automation is ESSENTIAL for regression testing. Manual regression on large systems is impractical.

Types of Regression Testing

- Full regression — run all test cases
- Selective regression — run tests related to changed code
- Progressive regression — add new tests for new features
- Re-test all — complete re-test after major refactoring
- Risk-based regression — prioritise by business impact
- Partial regression — subset of full regression suite

✚ 80% of defects in maintenance are regressions — previously existing functionality.
✚ 80% of maintenance defects are regressions — new changes breaking regressions.

4.9

Smoke and Sanity Testing

Quick health checks — before detailed testing begins.

Chapter 4 — Software Testing Fundamentals

Smoke vs Sanity Testing — Comparison

Both smoke and sanity testing are quick health checks — but they serve different purposes at different times:

Aspect	Smoke Testing	Sanity Testing
Definition	Preliminary check of critical functions of a new build to decide if it is stable enough for detailed testing	Narrow, focused check to verify a specific bug fix or new feature works correctly before full regression
When performed	After EVERY new build is received	After a specific bug fix or small change
Scope	BROAD and SHALLOW — covers all major features quickly	NARROW and DEEP — focuses on one specific area
Who performs	QA team (or automated CI pipeline)	QA team or developer who made the fix
Duration	Short (30–60 minutes typically)	Short (15–30 minutes typically)
Basis	Build release checklist, critical user journeys	Specific requirements for the change or fix made
If it fails	Build is rejected — not ready for testing	Change is sent back for rework — not deployed
Also called	Build verification test, BVT	Subset regression, targeted regression
Documentation	Smoke test checklist / test plan	Not always formally documented
Scripted/Exploratory	Usually scripted with defined test cases	Often exploratory and unscripted

4.10

Test Environment Setup

Creating the right conditions for reliable testing.

Chapter 4 — Software Testing Fundamentals

Section 4.10

Test Environments — Types & Purpose

A test environment is a combination of hardware, software, data, and configurations used to execute tests. Different environments serve different purposes:

Dev**Development Environment**

Developer's local machine. Used for unit testing and initial debugging. Not stable — changes frequently.

*Access: Developers only***INT****Integration Environment**

Shared environment where code from multiple developers is merged and tested together for integration testing.

*Access: Dev + QA teams***QA****QA/Test Environment**

Stable environment controlled by the QA team. Used for system testing, regression testing, and exploratory testing.

*Access: QA team***STG****Staging Environment**

Mirror of production environment with production-like data. Used for UAT, performance testing, and final validation.

*Access: QA + Business stakeholders***PROD****Production Environment**

Live environment serving real users. Testing in production only via canary releases, monitoring, and A/B testing.

Access: Operations / All users

Section 4.10

Test Environment Components & Setup

A complete test environment requires multiple carefully configured components:

Hardware

- Servers (physical or virtual cloud instances)
- Client machines (desktop, laptop, mobile devices)
- Network infrastructure (routers, switches, firewalls)
- Specialised hardware (IoT devices, embedded systems)

Software

- Operating systems (matching production versions)
- Application server, web server, middleware
- Databases with test data loaded
- Third-party services and integrations

Test Data

- Realistic data covering all test scenarios
- Edge case data for boundary testing
- Data masking/anonymisation for privacy compliance
- Data refresh procedures between test runs

Configuration

- Environment-specific configuration files
- Security certificates and credentials
- Network configuration and DNS settings
- Build and deployment scripts (IaC: Terraform, Ansible)



Best Practices for Test Environments:

- Use Infrastructure as Code (Terraform, Docker, Kubernetes) for reproducible environments
- Environment provisioning should be automated — no manual setup

4.11

Test Documentation

Test plan, test cases, test reports — IEEE 829 standard.

Section 4.11

Test Documentation — IEEE 829 Standard

IEEE Std 829 (Standard for Software and System Test Documentation) defines the required test documents:

1. Test Plan

TP

Master document describing test strategy, scope, resources, schedule, and approach. The cornerstone of all testing activities.

 *Written at start of project/sprint*

2. Test Design Specification

TDS

Describes the test conditions, test cases, test procedures, and expected results for a specific feature or module.

 *Before test execution begins*

3. Test Case Specification

TCS

Detailed test cases with test ID, preconditions, test steps, input data, expected results, and pass/fail criteria.

 *During test preparation phase*

4. Test Procedure Specification

TPS

Step-by-step instructions for executing test cases. Guides testers through exact testing procedures.

 *Before test execution*

5. Test Incident Report

TIR

Documents each defect found during testing — describes the issue, steps to reproduce, severity, priority, and screenshots.

 *During test execution — when defect found*

6. Test Summary Report

TSR

Summarises testing activities, coverage achieved, defects found, and overall quality assessment. Used for go/no-go decision.

 *At end of each test phase*

Section 4.11

Test Plan — Required Sections (IEEE 829)

A Software Test Plan must contain these sections as defined by IEEE 829 / IEEE 29119-3:

#	Section	Content Description
1	Test Plan Identifier	Unique identifier for the test plan document
2	Introduction	Purpose, scope, and overview of the testing effort
3	Test Items	Software and hardware to be tested (with versions)
4	Features to be Tested	Functions and requirements that WILL be tested
5	Features NOT to be Tested	Items explicitly excluded and justification
6	Approach	Overall strategy: levels, types, techniques to be used
7	Item Pass/Fail Criteria	Measurable entry and exit criteria for each test level
8	Suspension/Resumption	Conditions to halt and resume testing
9	Test Deliverables	Documents to be produced (test cases, reports, logs)
10	Testing Tasks	Specific testing activities, dependencies, and assignments
11	Environmental Needs	Hardware, software, test data, network requirements
12	Responsibilities	Who is responsible for each testing activity
13	Staffing and Training	Testing team skills and training needs
14	Schedule	Testing milestones, start/end dates, estimates

Test Case Specification — Template & Example

A well-written test case contains all information needed to execute and verify the test:

Field	Value	Description
Test Case ID	TC-LOGIN-001	Unique identifier for traceability
Test Case Name	Valid Login with Correct Credentials	Descriptive name of what is being tested
Module / Feature	Authentication — Login Form	Component being tested
Priority	High	Critical, High, Medium, Low
Requirement Ref	REQ-AUTH-001, REQ-AUTH-002	Links test to requirement (traceability)
Preconditions	User account exists. System is running. 1. Navigate to login page	Conditions that must be true before test executes
Test Steps	2. Enter valid username: 'admin' 3. Enter valid password: 'Admin@123'	Step-by-step instructions for test execution
Test Data	Username: admin Password: Admin@123	Input values to be used in the test
Expected Result	User is redirected to dashboard. Welcome message shows 'Welcome, Admin'. Session	What SHOULD happen if software is correct
Actual Result	(To be filled during execution)	What ACTUALLY happened
Pass/Fail	(To be filled during execution)	PASS / FAIL / BLOCKED / N/A
Post-condition	User is logged in. Dashboard is displayed.	State after test execution

Defect / Bug Report — Template & Severity Levels

A complete defect report enables developers to reproduce, understand, and fix bugs efficiently:

Defect ID:	BUG-2024-0142	Title:	Login fails with valid credentials after password change
Severity:	Critical	Priority:	P1 — Fix Immediately
Status:	Open	Reported By:	John Tester
Date Reported:	15/06/2024	Assigned To:	Dev Team Lead
Environment:	QA Environment v2.3.1	Build/Version:	Build 2.3.1-rc2

Defect Severity Levels:

Critical (S1)

System crash, data loss, security breach — complete show-stopper

High (S2)

Major feature not working, significant business impact

Medium (S3)

Feature partially broken, workaround available

Low (S4)

Minor UI issues, cosmetic defects, spelling errors

Chapter Summary

Key takeaways from Chapter 4.

Chapter 4 — Software Testing Fundamentals

Section 4.12

Chapter 4 — Key Concepts Summary

● Software Testing:	Process of executing software to find defects. Goals: find defects, verify requirements, validate needs, build confidence, measure quality.
● V&V:	Verification = 'Building it RIGHT?' (process checks). Validation = 'Building the RIGHT thing?' (user needs). Both are essential QA activities.
● 7 Testing Principles:	Testing shows presence of bugs · Exhaustive testing impossible · Early testing saves money · Defect clustering · Pesticide paradox · Context-dependent · Absence-of-errors fallacy.
● Test Pyramid:	Unit tests (many, fast, cheap) → Integration tests → System tests → Acceptance tests (few, slow, expensive). Optimal: 70/20/10 distribution.
● Black/White/Grey Box:	Black-box: tests from user view, no code knowledge. White-box: tests internal code paths. Grey-box: partial code knowledge for targeted testing.
● Functional Testing:	Tests WHAT system does. Key techniques: Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing.
● Non-Functional Testing:	Tests HOW WELL system performs. Types: Performance (load/stress/endurance/spike), Security, Usability, Reliability, Compatibility, Maintainability.
● Regression Testing:	Re-executes tests after changes to verify existing functionality is unbroken. Automation is essential. Types: full, selective, progressive, risk-based.
● Smoke vs Sanity:	Smoke: broad/shallow check of new build stability before detailed testing. Sanity: narrow/deep check of specific fix/feature correctness.
● Test Environments:	Dev → Integration → QA → Staging → Production. Each serves different purpose. Should be isolated, automated (IaC), and version-controlled.
● Test Documentation:	IEEE 829 defines: Test Plan, Test Design Spec, Test Case Spec, Test Procedures, Test Incident Report, Test Summary Report.
● Defect Severity:	Critical (S1: crash/data loss) · High (S2: major feature broken) · Medium (S3: partial with workaround) · Low (S4: cosmetic/minor issues).

Section 4.6

Equivalence Partitioning & BVA — Worked Examples

Worked Example: A student grade entry system accepts marks from 0 to 100 (integer):

Equivalence Partitioning:

Partition	Range	Type
EP1	Below 0	Invalid
EP2	0 to 100	Valid
EP3	Above 100	Invalid
EP4	Non-integer	Invalid
EP5	Empty/null	Invalid

✂ EP reduces 101 valid inputs + infinite invalid inputs to just 5 representative test cases — making testing practical and efficient.

Boundary Value Analysis:

Boundary	Test Values	Why
Lower boundary	-1, 0, 1	Just below, at, just above mi
Upper boundary	99, 100, 101	Just below, at, just above ma
Nominal	50	Mid-range value

☑ BVA Test Values: -1, 0, 1, 50, 99, 100, 101
(7 test cases cover all boundaries)

✂ Most defects occur at boundary values — BVA targets these high-risk points systematically.

Decision Table Testing — Example

Decision table testing: systematically cover all combinations of conditions. Example: Loan approval system:

Conditions: (1) Income > 50,000 (2) Credit Score > 700 (3) Employment = Employed

Condition/Action	TC1	TC2	TC3	TC4	TC5	TC6	TC7
Income > 50K	T	T	T	T	F	F	F
Credit > 700	T	T	F	F	T	T	F
Employed	T	F	T	F	T	F	T
—————	—	—	—	—	—	—	—
Approve Loan	✓	✗	✗	✗	✗	✗	✗
Reject Loan		✓	✓	✓	✓	✓	✓
Request Info							



Decision table testing ensures all combinations are considered. With 3 binary conditions there are $2^3 = 8$ possible combinations. TC1 is the only approval case — all other combinations lead to rejection.

White-Box Testing — Coverage Criteria

White-box coverage criteria define how thoroughly the internal code structure has been tested:

1	Statement Coverage (C0) Formula: <code>Statements executed / Total statements × 100%</code>	Every executable statement in the code is executed at least once. Minimum acceptable coverage. Finds uncovered dead code.	Target: 80%+
2	Branch Coverage (C1) Formula: <code>Branches executed / Total branches × 100%</code>	Every branch (both TRUE and FALSE) of every decision point is executed. More thorough than statement coverage.	Target: 70%+
3	Condition Coverage (C2) Formula: <code>Conditions evaluated T and F / Total conditions × 2</code>	Each boolean sub-condition in every decision is evaluated as both TRUE and FALSE at least once.	Target: 70%+
4	Path Coverage (C3) Formula: <code>Paths covered / Total independent paths × 100%</code>	Every independent execution path from entry to exit is executed. Most thorough but impractical for large systems.	Target: Key paths
5	Modified Condition/Decision Coverage (MC/DC) Formula: <code>Complex formula based on independent conditions</code>	Each condition independently affects the decision outcome. Required for aviation (DO-178C Level A), automotive, medical software.	Target: 100% (safety critical)

Higher coverage ≠ more quality. 100% statement coverage can still miss many defects. Use coverage as a floor, not a ceiling. Focus on risk-

Test Automation — Overview & Best Practices

Test automation uses software tools to execute tests automatically, compare results, and report outcomes:

When to Automate

- Regression tests run frequently
- Repetitive, time-consuming tests
- Tests requiring multiple data sets
- Performance/load testing
- API testing and integration tests
- Cross-browser compatibility tests

When NOT to Automate

- One-time tests (not worth investment)
- Exploratory and usability testing
- Tests with frequently changing UI
- Highly complex user interaction tests
- Tests requiring human judgement
- Initial feasibility testing

Popular Test Automation Tools:

Key Terms & Definitions — Chapter 4

Software Testing	Systematic process of executing software to find defects and verify it meets requirements.
Verification	Confirming the product is built RIGHT — conforms to specifications (process-focused).
Validation	Confirming the RIGHT product is built — meets user needs (product/user-focused).
Unit Testing	Testing individual software components (functions/classes) in isolation from the rest.
Integration Testing	Testing interactions and interfaces between integrated software components.
System Testing	Testing the complete integrated system against specified functional requirements.
Acceptance Testing	Testing by users/customers to validate the system meets their business needs.
Black-box Testing	Testing without knowledge of internal code — only inputs and outputs considered.
White-box Testing	Testing with full knowledge of internal code structure and execution paths.
Regression Testing	Re-testing unchanged software to ensure changes haven't broken existing functionality.
Smoke Testing	Quick broad test of a new build to verify it is stable enough for detailed testing.
Sanity Testing	Narrow test of a specific bug fix or change to verify it works correctly.
Test Plan	Document describing scope, approach, resources, and schedule for testing activities (IEEE 829).
Test Case	Documented input conditions, expected results, and pass/fail criteria for a specific test.
Defect	A flaw in a software product that causes it to fail to perform its required function.
Severity	The impact of a defect on system functionality (Critical, High, Medium, Low).
Priority	The urgency to fix a defect (P1: Fix immediately → P4: Fix in future release).

Review & Practice Questions

-
- Q1** Define software testing. What are the six goals of software testing? Why can't testing prove the absence of defects?
- Q2** Explain the difference between Verification and Validation. Give two examples of each. Which comes first?
- Q3** List and explain all seven principles of software testing (ISTQB). Which do you think is most important and why?
- Q4** Describe the four levels of software testing. Draw and label the test pyramid showing optimal test distribution.
- Q5** Compare black-box, white-box, and grey-box testing. When would you use each? Give examples of techniques for each.
- Q6** Describe four functional testing techniques (EP, BVA, Decision Table, State Transition) with worked examples.
- Q7** List six types of non-functional testing. For performance testing, describe load, stress, endurance, and spike testing.
- Q8** What is regression testing? List the types of regression testing and explain when each should be used.
- Q9** Compare smoke testing and sanity testing. When is each conducted, who performs them, and what distinguishes them?
- Q10** Describe the five test environments (Dev → Production). What is the purpose of each? Why must they be isolated?
- Q11** List the six documents defined in IEEE 829. Describe the required sections of a Software Test Plan.
- Q12** Design a complete test case specification for testing a password reset feature in a banking application.

Chapter 4 — Software Testing Fundamentals — Quick Reference Card

Print this slide — all essential testing knowledge in one place

Test Levels

1. Unit Test (isolation)
 2. Integration Test
 3. System Test
 4. Acceptance Test
- Pyramid: 70/20/10
Unit → fastest, cheapest
UAT → slowest, costly

Test Types

Black-box: external view
White-box: internal code
Grey-box: partial view
EP + BVA: functional
Decision table: logic
State transition: flows
Exploratory: unscripted

Non-Functional

Performance (load/stress)
Security (SAST/DAST/pen)
Usability (UX/a11y)
Reliability (MTBF/MTTR)
Compatibility (browsers)
Maintainability (SonarQ)
Scalability testing

Regression

After EVERY code change
Full / Selective / Partial
Risk-based prioritisation
Automate all regression
Smoke: broad, pre-test
Sanity: narrow, post-fix
80% maint bugs = regressions

Test Docs (IEEE 829)

Test Plan (TP)
Test Design Spec (TDS)
Test Case Spec (TCS)
Test Procedures (TPS)
Test Incident Report
Test Summary Report
Defect: S1→S4 severity

7 ISTQB Principles

1. Shows presence of bugs
2. Exhaustive = impossible
3. Early testing saves \$
4. Defect clustering (80/20)
5. Pesticide paradox
6. Context-dependent
7. Absence-of-errors fallacy

Exam Tips — Chapter 4

7 Testing Principles (Must Memorise)

- Memorise all 7 ISTQB principles with their names
- Know which principle: Dijkstra quote → 'shows presence of bugs'
- Defect clustering = 80/20 rule (Pareto principle) applied to testing
- Pesticide paradox = same tests = pesticide resistance = new bugs escape

V&V and Test Types (Know Well)

- V = 'building it RIGHT' (process), V = 'building RIGHT thing' (product)
- BB/WB/Grey-box: when to use each and their techniques
- 4 functional techniques: EP, BVA, Decision Table, State Transition
- 4 performance types: Load, Stress, Endurance, Spike

Test Levels & Pyramid (Must Know)

- 4 levels: Unit → Integration → System → Acceptance (UAT)
- Test pyramid: 70% unit, 20% integration, 10% E2E
- Who performs each: developers/QA/users
- V-Model pairing: which level maps to which dev phase

Test Documentation (IEEE 829)

- 6 documents: TP, TDS, TCS, TPS, TIR, TSR
- Test Plan has 15 required sections — know the key ones
- Test case structure: ID, preconditions, steps, expected, actual, pass/fail
- Defect severity: Critical(S1), High(S2), Medium(S3), Low(S4)

Practice Exercises — Chapter 4

Exercise 1

Test Case Design — EP & BVA

A registration form accepts: Age (18–65), Username (6–15 characters, alphanumeric), Password (8+ chars, must contain uppercase, digit, special char). Design test cases using both Equivalence Partitioning and Boundary Value Analysis for all three fields. Create a complete test case table.

Exercise 2

Decision Table Testing

An online discount system grants discounts based on: (a) Customer type: Premium/Regular, (b) Purchase amount: >\$100 or ≤\$100, (c) Voucher: Valid/Invalid. Design a complete decision table covering all 8 combinations. Define the expected discount for each combination.

Exercise 3

Test Plan Draft

Draft a one-page Test Plan (following IEEE 829) for a banking mobile app. Include: scope (what to test and what not to test), testing levels, types of testing required, entry/exit criteria for each phase, team responsibilities, schedule with milestones, and major risks with mitigation strategies.

Exercise 4

Defect Report Writing

A tester discovers that: When a user with admin role deletes another user's account, the deleted user's sessions remain active and they can continue to use the system indefinitely. Write a complete defect report including: severity (and justify), priority, steps to reproduce, expected vs actual results, and suggested fix.

Exercise 5

Test Environment Design

Design a complete test environment architecture for an e-commerce platform. Specify: all required environments (Dev through Production), the purpose and configuration of each, test data strategy (what data in each environment), tools to be used, and how environment provisioning will be automated.

References & Further Reading — Chapter 4

- **Myers, G.J., Sandler, C., & Badgett, T. (2011)**
The Art of Software Testing (3rd ed.). Wiley. [The definitive testing textbook]
- **ISTQB Certified Tester Foundation Level (2018)**
ISTQB CT-FL Syllabus v3.1. International Software Testing Qualifications Board. [istqb.org](https://www.istqb.org)
- **IEEE Std 829-2008 (2008)**
IEEE Standard for Software and System Test Documentation. IEEE Computer Society.
- **Beizer, B. (1990)**
Software Testing Techniques (2nd ed.). Van Nostrand Reinhold. [Classic techniques reference]
- **Cohn, M. (2009)**
Succeeding with Agile: Software Development Using Scrum. Addison-Wesley. [Test pyramid origin]
- **Fowler, M. (2018)**
"On the Testing Pyramid." martinfowler.com/bliki/TestPyramid.html
- **Kaner, C., Bach, J., & Pettichord, B. (2001)**
Lessons Learned in Software Testing: A Context-Driven Approach. Wiley.
- **Pressman, R.S. & Maxim, B.R. (2019)**
Software Engineering: A Practitioner's Approach (9th ed.), Chapter 22–24. McGraw-Hill.
- **OWASP Testing Guide (2023)**
The OWASP Web Security Testing Guide v4.2. OWASP Foundation. owasp.org
- **Fagan, M.E. (1976)**
"Design and Code Inspections to Reduce Errors in Program Development." IBM Systems Journal, 15(3).

Software Testing Tools Reference Guide

Comprehensive reference of testing tools by category:

Unit Testing

- JUnit 5 — Java unit testing framework
- pytest — Python testing framework
- NUnit — .NET unit testing
- Jest — JavaScript unit + mocking
- xUnit — .NET alternative to NUnit
- Mockito — Java mocking framework

UI / E2E Testing

- Selenium WebDriver — cross-browser automation
- Cypress — modern E2E testing framework
- Playwright — Microsoft multi-browser E2E
- Appium — mobile app automation
- TestCafe — no-driver web testing
- Robot Framework — keyword-driven testing

API Testing

- Postman — API testing and documentation
- REST Assured — Java REST API testing
- Karate DSL — API test automation
- SoapUI — SOAP/REST API testing
- Pact — contract testing
- Newman — Postman CLI runner

Performance Testing

- Apache JMeter — open-source load testing
- Gatling — Scala-based performance tool
- Locust — Python load testing tool
- k6 — modern JavaScript-based load tool
- BlazeMeter — cloud performance testing
- Artillery — Node.js load testing

Security Testing

- OWASP ZAP — free DAST scanner
- Burp Suite — web app security testing
- Nessus — vulnerability assessment
- Metasploit — penetration testing
- SonarQube — SAST and code quality
- Snyn — dependency vulnerability scanning

Test Management

- Jira + Zephyr — test case management
- TestRail — comprehensive TMS
- qTest — enterprise test management
- PractiTest — end-to-end visibility
- Azure Test Plans — Microsoft TMS
- Xray for Jira — test management plugin

Tool selection depends on: language/framework, team skills, budget, CI/CD integration requirements, and project type. Open-source tools

Testing Strategies in Agile Development

Agile testing integrates quality activities continuously throughout each sprint:

Activity	When in Sprint	QA Role and Deliverable
Sprint Planning	Day 1	Define acceptance criteria for each story · Estimate test effort · Identify test data needs
Story refinement	Day 1–2	Review stories for testability · Clarify ambiguous requirements · Define DoR criteria
Test case design	Days 1–3	Write test cases based on acceptance criteria · Identify test data and environments
Unit testing (TDD)	Days 1–8 (continuous)	Developer writes failing test · Developer codes to pass test · QA reviews coverage
Exploratory testing	Days 3–8	Unscripted testing beyond defined test cases · Find edge cases and usability issues
Integration testing	Days 4–9 (CI/CD)	Automated on every commit · Integration test suite run automatically · Failures halt pipeline
Regression testing	Days 5–9 (automated)	Full automated regression suite run · Results reviewed · New failures investigated
UAT / Review	Day 10	Demo to stakeholders · Acceptance criteria verified · Product Owner sign-off
Sprint retrospective	Day 10 (end)	Review testing process · Identify improvements · Update DoD if needed

State Transition Testing — Detailed Example

State Transition Testing maps system states, events, transitions, and actions. Example: ATM Card States:

Idle

Transitions:

- Card inserted → Card Reading

Card Reading

Transitions:

- Valid card → PIN Entry
- Invalid card → Card Ejected

PIN Entry

Transitions:

- Correct PIN → Menu Shown
- Wrong PIN (1st) → PIN Entry
- Wrong PIN (3rd) → Card Seized

Menu Shown

Transitions:

- Transaction selected → Processing
- Cancel → Card Ejected

Processing

Transitions:

- Success → Idle (receipt printed)
- Failure → Error State

Error State

Transitions:

- Timeout → Idle
- Operator reset → Idle

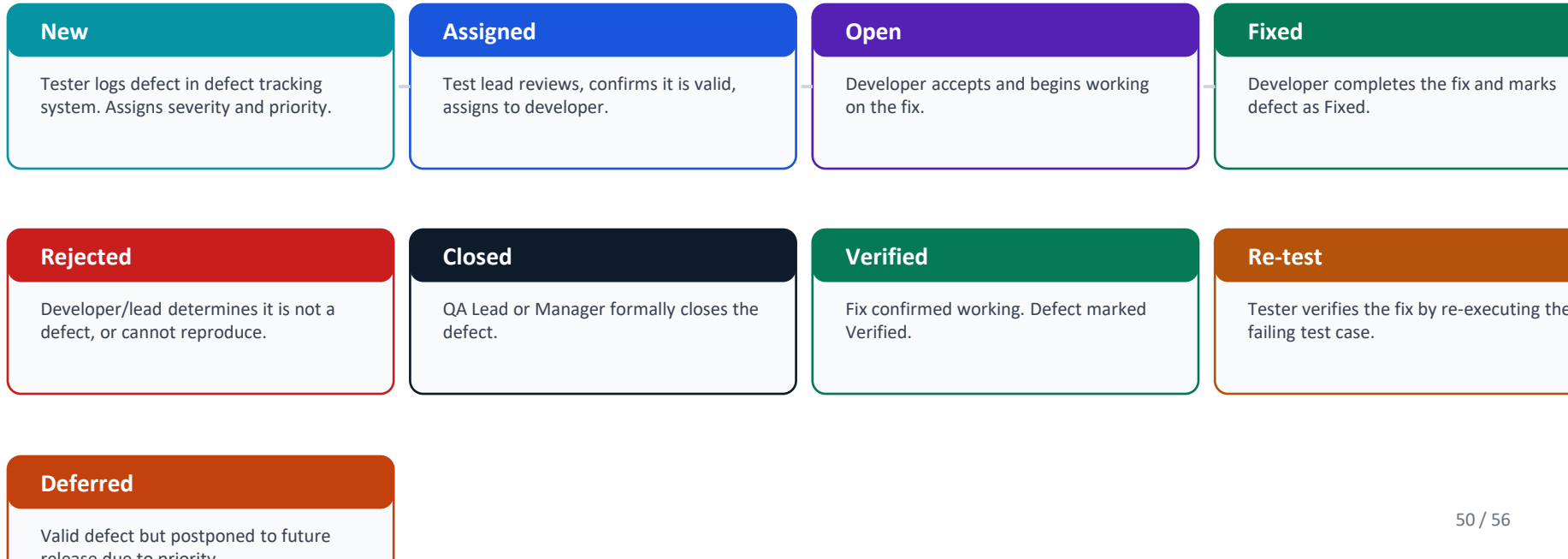
Key Testing Metrics Reference

Measuring testing effectiveness and efficiency — key metrics every QA engineer should know:

Metric	Formula	Target	What it Measures
Defect Density	Defects / KLOC	< 1 per KLOC	Code quality and maturity
Defect Detection Rate	Defects found / Sprint	Decreasing trend	Testing thoroughness over time
Defect Removal Efficiency	$(\text{Pre-release defects} / \text{Total defects}) \times 100$	≥ 95%	Test effectiveness in finding defects early
Test Coverage	$(\text{Test cases executed} / \text{Total planned}) \times 100$	100% of critical	Completeness of test execution
Code Coverage	$(\text{Statements executed} / \text{Total}) \times 100$	≥ 80%	Code exercised by tests
Test Case Pass Rate	$(\text{Passed} / \text{Total executed}) \times 100$	≥ 90% for release	Current quality of build
Defect Reopen Rate	$(\text{Reopened defects} / \text{Closed defects}) \times 100$	< 5%	Fix quality and regression rate
Mean Time To Detect	Time from defect intro to discovery	Minimise	Testing lag — how fast we find bugs
Test Execution Rate	Test cases run per day	Per plan schedule	Team testing velocity
Requirements Coverage	$(\text{Requirements tested} / \text{Total}) \times 100$	100% critical, 80%+ all	Functional completeness
Automation Coverage	$(\text{Automated TCs} / \text{Total TCs}) \times 100$	≥ 70% regression	Automation maturity
Escaped Defect Rate	Production defects / Releases	< 0.1 per release	Testing effectiveness overall

Defect / Bug Lifecycle — States & Workflow

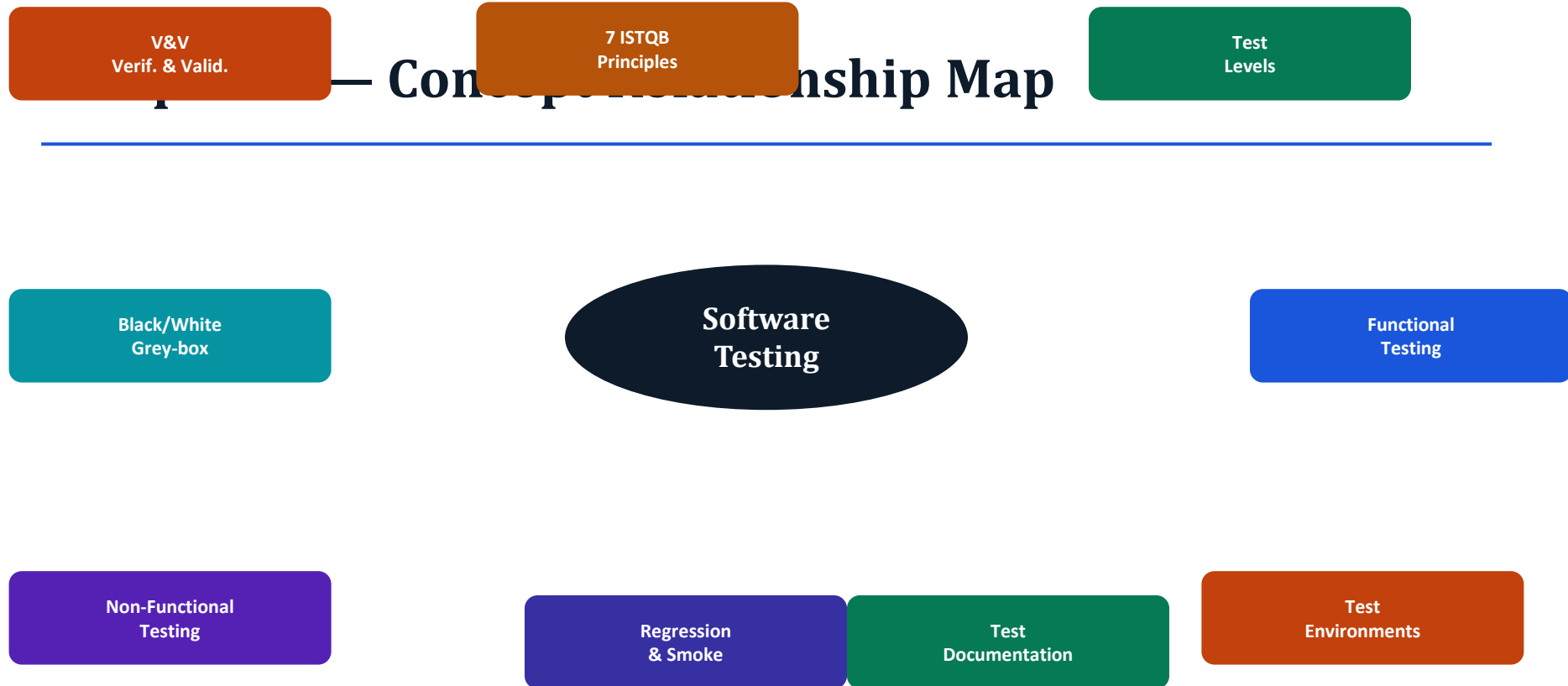
Every defect progresses through a defined lifecycle from discovery to closure:



Alpha Testing vs Beta Testing

Two forms of Acceptance Testing — both test software in realistic conditions before public release:

Aspect	Alpha Testing	Beta Testing
Definition	Testing by internal employees/team in a controlled environment simulating real user behaviour	Testing by actual end-users in their own real-world environment before public release
Who performs	Internal team members, selected employees	External end-users (real customers)
Location	Developer's site / controlled lab environment	Customer's own environment
When	Before beta testing — near end of development	After alpha testing — just before general release
Environment	Simulated / controlled test environment	Real production-like environment
Purpose	Find defects before external release	Get real-world feedback and find remaining bugs
Feedback	Immediate — developer/tester can ask questions	Delayed — users report issues via bug reports
Attitude	Testers may know system — less unbiased	Real users — genuine unbiased reactions
Duration	Days to weeks	Weeks to months (depends on product)
Examples	Microsoft dogfooding · Internal UAT	Google Beta · iOS TestFlight · Steam Early Access



Testing Levels — Complete Reference Comparison

Level	Who	When	Approach	Tools
Unit	Developers	During coding	White-box · TDD	JUnit, pytest, Jest
Integration	Dev/QA	After unit test	Black/Grey-box · Top-down/Bottom-up	Postman, REST Assured
System	QA team	After integration	Black-box · All techniques	Selenium, Cypress, TestNG
Performance	Perf team	After system test	Non-functional · Load/Stress	JMeter, Gatling, k6
Security	Security team	After system test	SAST/DAST/Pen test	ZAP, Burp Suite, Nessus
UAT	End users	Pre-release	Black-box · Scenario-based	TestRail, manual scripts
Regression	QA team	After any change	Automated · Risk-based	Selenium, Cypress, Robot
Smoke	QA/CI pipeline	Each new build	Broad/Shallow · Scripted	Jenkins, GitHub Actions
Sanity	QA team	After fix/change	Narrow/Deep · Exploratory	Manual or automated
Alpha	Internal team	Pre-beta	Exploratory · Scenario-based	Manual + automation
Beta	End users	Pre-release	Real-world · Exploratory	Bug reporting tools

Requirements Traceability Matrix (RTM) — Example

An RTM links each requirement to its design element, code module, and test cases — ensuring complete test coverage:

Req ID	Requirement	Design Ref	Code Module	TC ID	Status	Priority
REQ-001	User can log in with valid credentials	SDD-3.1	auth.login()	TC-001	Passed	High
REQ-002	Login fails with invalid password	SDD-3.2	auth.validate()	TC-002	Passed	High
REQ-003	Account locks after 3 failed attempts	SDD-3.3	auth.lockout()	TC-003	Failed	Critical
REQ-004	Password reset email sent within 2min	SDD-3.4	auth.reset()	TC-004	Passed	High
REQ-005	Session expires after 30 min inactivity	SDD-3.5	session.mgr()	TC-005	Not Run	Medium
REQ-006	Password must meet complexity rules	SDD-3.6	auth.policy()	TC-006	Passed	High
REQ-007	Multi-factor authentication option	SDD-3.7	auth.mfa()	TC-007	Blocked	High
REQ-008	Audit log captures all login events	SDD-3.8	audit.log()	TC-008	Passed	Medium



RTM shows: REQ-003 (Critical!) is FAILED — cannot release until this is fixed. REQ-005 is 'Not Run' — need to schedule. REQ-007 is

Test Strategy vs Test Plan — Key Differences

Students often confuse Test Strategy and Test Plan. Here is the definitive distinction:

Aspect	Test Strategy	Test Plan
Level	Organisation/programme level	Project level
Scope	Applies to all projects in organisation	Specific to ONE project
Created by	QA Manager / Test Director	Test Lead / QA Engineer
When created	Before projects begin — permanent	At start of each project — evolves
Content	Testing standards, methods, tools, processes	Scope, schedule, resources, risks for THIS project
Derived from	Organisation's QA policy	Test Strategy + Project requirements
Flexibility	Rarely changes	Updated throughout project
Purpose	Defines HOW to test in general	Defines HOW to test THIS project
Standard	IEEE 829 Section 8	IEEE 829 (entire document)
Example content	Always use automated regression · Use Agile Quadrants All test cases must be traceable to requirements	Test login module by 15/06 · 3 testers assigned · Risk: test login module by 15/06 · 3 testers assigned · Risk: test login module by 15/06 · 3 testers assigned · Risk: